

Controller

Controller層の責務

Controller層は、HTTP requestを受け取り、Usecaseへ渡せるInput DTOに変換し、Usecaseの結果をHTTP responseへ変換する。

ControllerはEcho Contextを扱ってよいが、UsecaseにはEcho Contextを渡さない。

Usecaseへ渡すのは、`context.Context` とInput DTOのみとする。

Controllerには、保存可否、評価可否、ランキング計算、モーダル表示可否、DB存在確認、権限判定などの業務ロジックを書かない。

項目	Controllerでやる	Controllerでやらない
Request取得	JSON body、Path Param、Query Param、Header、Cookieを取得する	業務判断はしない
DTO変換	HTTP requestをUsecase Inputへ変換する	Entityを直接組み立てすぎない
AppContext取得	Middlewareが保存したUserID、Role、GuestSessionID、RequestIDを取得する	user_idをRequest Bodyから信用しない
Usecase呼び出し	<code>ctx</code> とInput DTOを渡してUsecaseを呼ぶ	Repositoryを直接呼ばない
Response変換	Usecase OutputをHTTP JSONへ変換する	UsecaseにHTTP statusを返させない
Error変換	Usecase errorをHTTP status / error responseへ変換する	Repository errorをそのまま外へ出さない
Cookie操作	refresh/logout/csrfなどHTTP都合のCookieを扱う	RefreshToken rotationやreuse検知はしない
Header設定	CSRF token発行、RateLimit結果、RequestIDなどを返す	業務ログ・監査ログの作成はUsecase側

Controller共通ルール

ルール	内容
Echo Context	Controller内だけで使う
<code>context.Context</code>	<code>c.Request().Context()</code> から取り出してUsecaseへ渡す
AppContext	Middlewareで確定したUserID/GuestSessionID/RequestIDを取り出す
Validation	Request DTOの形式検証はValidatorまたはUsecase経由で行う
Business Logic	Controllerに書かない
Repository	Controllerから直接呼ばない
HTTP Status	ControllerでUsecase errorをHTTP statusへ変換する
any型	使わない。AppContext取得用の関数で型を隠蔽する

Controller処理の手順

順番	処理	内容
1	<code>ctx</code> 取得	<code>ctx := c.Request().Context()</code> を取得する
2	AppContext取得	UserID、Role、GuestSessionID、RequestIDを取得する
3	Request取得	Body、Path Param、Query Param、Cookie、Headerを取得する
4	Input DTO作成	Usecaseへ渡すInputに変換する
5	Usecase呼び出し	<code>usecase.Execute(ctx, input)</code> を呼ぶ
6	Error変換	Usecase errorをHTTP statusとerror responseに変換する
7	Response変換	Usecase OutputをHTTP JSONへ変換する
8	Response返却	<code>c.JSON(status, response)</code> で返す

Controllerで使う共通DTORequest DTO

Request DTOは、HTTP requestを受け取るための構造体。

ControllerはRequest DTOをUsecase Inputへ変換。

種類	例	役割
Request DTO	<code>SignupRequest</code>	HTTP request bodyを受け取る
Usecase Input	<code>SignupInput</code>	Usecaseへ渡す入力
Usecase Output	<code>LoginOutput</code>	Usecaseから返る結果
Response DTO	<code>LoginResponse</code>	HTTP responseとして返す形

AuthController

AuthControllerは、会員登録、ログイン、AccessToken再発行、ログアウト、ログイン中ユーザー取得。

RefreshTokenの検証、rotation、reuse検知、token_version更新はAuthUsecaseの責務であり、Controllerには書かない。

Controller	関数名	HTTP	認証	CSRF	主なUsecase	役割
<code>AuthController</code>	<code>Signup</code>	<code>POST /auth/signup</code>	不要	不要	<code>AuthUsecase.Signup</code>	会員登録
<code>AuthController</code>	<code>Login</code>	<code>POST /auth/login</code>	不要	不要	<code>AuthUsecase.Login</code>	ログイン
<code>AuthController</code>	<code>Refresh</code>	<code>POST /auth/refresh</code>	不要	必須	<code>AuthUsecase.RefreshAccessToken</code>	AccessToken再発行
<code>AuthController</code>	<code>Logout</code>	<code>POST /auth/logout</code>	必須	必須	<code>AuthUsecase.Logout</code>	ログアウト
<code>AuthController</code>	<code>Logout</code>	<code>POST /auth/logout-all</code>	必須	必須	<code>AuthUsecase.LogoutAll</code>	全RefreshTokenを失効しAccessToken無効化する
<code>AuthController</code>	<code>Me</code>	<code>GET /auth/me</code>	必須	不要	<code>AuthUsecase.GetCurrentUser</code>	ログイン中ユーザー取得

Signup

項目	内容
Request	<code>SignupRequest</code>
Input	<code>SignupInput</code>
Response	<code>SignupResponse</code>
成功時	<code>201 Created</code>
失敗例	validation error, email already exists, rate limited

Login

項目	内容
Request	<code>LoginRequest</code>
Input	<code>LoginInput</code>
Response	<code>LoginResponse</code>
Cookie	<code>refresh_token</code> , <code>csrf_token</code> を設定する
成功時	<code>200 OK</code>
失敗例	invalid credentials, user suspended, rate limited

Refresh

項目	内容
Request	Cookieから <code>refresh_token</code> 、Headerから <code>X-CSRF-Token</code>

項目	内容
Input	<code>RefreshAccessTokenInput</code>
Response	<code>RefreshAccessTokenResponse</code>
Cookie	新しい <code>refresh_token</code> と <code>csrf_token</code> を設定する
成功時	<code>200 OK</code>
失敗例	<code>csrf error</code> , <code>refresh token expired</code> , <code>reuse detected</code>

Logout

項目	内容
Request	認証済みUserID、Cookieの <code>refresh_token</code>
Input	<code>LogoutInput</code>
Response	なし
Cookie	<code>refresh_token</code> と <code>csrf_token</code> を削除する
Usecase	<code>AuthUsecase.LogoutCurrentFamily</code>
成功時	<code>204 No Content</code>
失敗例	<code>unauthorized</code> , <code>csrf error</code> , <code>rate limited</code>

通常Logoutでは、現在の `refresh token family` のみを失効する。

`users.token_version` は増やさない。

フロントはメモリ上の `AccessToken` 破棄する。

LogoutAll

項目	内容
Request	認証済みUserID、Cookieの <code>refresh_token</code>
Input	<code>LogoutAllInput</code>
Response	なし
Cookie	<code>refresh_token</code> と <code>csrf_token</code> を削除する
Usecase	<code>AuthUsecase.LogoutAllDevices</code>
成功時	<code>204 No Content</code>
失敗例	<code>unauthorized</code> , <code>csrf error</code> , <code>rate limited</code>

LogoutAllでは、Userの全 `refresh token` を失効し、`users.token_version` を増やす。

これにより、既存の `AccessToken` も無効化する。

Me

項目	内容
Request	AppContextのUserID
Input	<code>GetCurrentUserInput</code>
Response	<code>MeResponse</code>
成功時	<code>200 OK</code>
失敗例	<code>unauthorized</code> , <code>user not found</code>

CSRFController

CSRFControllerは、Double Submit Cookie用のCSRF tokenを発行。

CSRF token生成とCookie設定はHTTP都合の処理であるためControllerで扱う。

Controller	関数名	HTTP	認証	CSRF	役割
<code>CSRFController</code>	<code>Issue</code>	<code>GET /auth/csrf</code>	不要	不要	CSRF tokenを発行する

項目	内容
Response	<code>CSRFResponse</code>
Cookie	<code>csrf_token</code> を設定する
成功時	200 OK
注意	<code>csrf_token</code> はJavaScriptからHeaderへ載せる必要があるためHttpOnlyにしない

BeanController

BeanControllerは、公開済みBeanの一覧・詳細を返す。

ランキング順、検索条件、公開状態の判断はUsecaseで行う。

Controller	関数名	HTTP	認証	主なUsecase	役割
<code>BeanController</code>	<code>ListBeans</code>	GET /beans	不要	<code>BeanUsecase.ListBeans</code>	Bean一覧
<code>BeanController</code>	<code>GetBeanDetail</code>	GET /beans/:id	不要	<code>BeanUsecase.GetBeanDetail</code>	Bean詳細

ListBeans

項目	内容
Query	<code>limit</code> , <code>offset</code> , <code>sort</code> , <code>origin</code> , <code>roast_level</code> など
Input	<code>ListBeansInput</code>
Response	<code>BeanListResponse</code>
成功時	200 OK
失敗例	invalid query, rate limited

GetBeanDetail

項目	内容
Path Param	<code>id</code>
Input	<code>GetBeanDetailInput</code>
Response	<code>BeanDetailResponse</code>
成功時	200 OK
失敗例	invalid id, bean not found

SearchController

SearchUsecase内でSearchValidatorを呼び出す。

SearchValidator、Repository、GORM、DB、Redis に依存しない。

Query Param、Cookie、ApplicationContextをUsecase Inputへ変換して SearchUsecaseを呼ぶだけにする。

Controller	関数名	HTTP	認証	CSRF	主なUsecase	主ない
<code>SearchController</code>	<code>SearchBeans</code>	GET /search/beans	UserまたはGuest	不要	<code>SearchUsecase.SearchBeans</code>	Sear
<code>SearchController</code>	<code>SearchArticles</code>	GET /search/articles	UserまたはGuest	不要	<code>SearchUsecase.SearchArticles</code>	Sear

SearchControllerの責務

項目	Controllerでやる	Controllerでやらない
Request取得	Query Paramを取得する	検索条件の業務判断はしない

項目	Controllerでやる	Controllerでやらない
AppContext取得	Middlewareが確定したUserID / GuestSessionID / RequestIDを取得する	user_id や guest_session_id を Request Bodyから受け取らない
DTO変換	Query Paramを SearchBeansInput / SearchArticlesInput に変換する	Entityを直接組み立てない
Validation	Controllerでは直接Validatorを呼ばない。SearchUsecaseがSearchValidatorを呼ぶ	DB存在確認、検索結果有無は見ない
Usecase呼び出し	SearchUsecaseを呼ぶ	Repositoryを直接呼ばない
Response変換	Usecase OutputをJSON Responseに変換する	Ranking score計算はしない
Error変換	Usecase errorをHTTP statusへ変換する	Repository errorをそのまま返さない
依存	SearchUsecaseのみ	SearchValidator, Repository, GORM, DB, Redis

SearchController は SearchUsecase に依存する。
SearchController は SearchValidator、Repository、GORM、DB、Redisに直接依存しない。
SearchValidatorはSearchUsecaseから呼び出す。

SearchBeans

項目	内容
Controller	<code>SearchController</code>
関数名	<code>SearchBeans</code>
HTTP	<code>GET /search/beans</code>
認証	UserまたはGuest
CSRF	不要
Query	<code>q</code> , <code>roast_level</code> , <code>origin</code> , <code>acidity</code> , <code>bitterness</code> , <code>flavor</code> , <code>aroma</code> , <code>body</code> , <code>sort</code> , <code>limit</code> , <code>offset</code>
Request DTO	<code>SearchBeansQuery</code>
Input	<code>SearchBeansInput</code>
Response	<code>SearchBeanListResponse</code>
Usecase	<code>SearchUsecase.SearchBeans</code>
Validator	<code>SearchValidator.ValidateSearchBeansQuery</code>
成功時	<code>200 OK</code>
失敗例	<code>invalid query</code> , <code>actor required</code> , <code>rate limited</code>

SearchArticles

項目	内容
Controller	<code>SearchController</code>
関数名	<code>SearchArticles</code>
HTTP	<code>GET /search/articles</code>
認証	User または Guest
CSRF	不要
Query	<code>q</code> , <code>category</code> , <code>sort</code> , <code>limit</code> , <code>offset</code>
Request DTO	<code>SearchArticlesQuery</code>
Input	<code>SearchArticlesInput</code>
Response	<code>SearchArticleListResponse</code>
Usecase	<code>SearchUsecase.SearchArticles</code>
Validator	<code>SearchValidator.ValidateSearchArticlesQuery</code>
成功時	<code>200 OK</code>
失敗例	<code>invalid query</code> , <code>actor required</code> , <code>rate limited</code>

SearchArticlesQuery

項目	型	許可値	デフォルト	Controllerでやること
q	string	1~100文字	なし	Queryから取得してDTOへ詰める
category	string	brewing, roast, beans, recipe	なし	Queryから取得してDTOへ詰める
sort	string	score, newest, popular	score	未指定ならデフォルト値を入れる
limit	int	1~50	20	未指定ならデフォルト値を入れる
offset	int	0~500	0	未指定ならデフォルト値を入れる

SearchControllerで扱うQuery項目SearchBeansQuery

項目	型	許可値	デフォルト	Controllerでやること
q	string	1~100文字	なし	Queryから取得してDTOへ詰める
roast_level	string	light, medium, dark	なし	Queryから取得してDTOへ詰める
origin	string	1~50文字	なし	Queryから取得してDTOへ詰める
acidity	int	1~5	なし	数値変換してDTOへ詰める
bitterness	int	1~5	なし	数値変換してDTOへ詰める
flavor	int	1~5	なし	数値変換してDTOへ詰める
aroma	int	1~5	なし	数値変換してDTOへ詰める
body	int	1~5	なし	数値変換してDTOへ詰める
sort	string	score, newest, popular	score	未指定ならデフォルト値を入れる
limit	int	1~50	20	未指定ならデフォルト値を入れる
offset	int	0~500	0	未指定ならデフォルト値を入れる

SearchArticlesQuery

項目	型	許可値	デフォルト	Controllerでやること
q	string	1~100文字	なし	Queryから取得してDTOへ詰める
category	string	brewing, beans, equipment, news	なし	Queryから取得してDTOへ詰める
tag	string	1~30文字	なし	Queryから取得してDTOへ詰める
sort	string	score, newest, popular	score	未指定ならデフォルト値を入れる
limit	int	1~50	20	未指定ならデフォルト値を入れる
offset	int	0~500	0	未指定ならデフォルト値を入れる

SearchController は SearchUsecase に依存する。

SearchController は SearchValidator、Repository、GORM、DB、Redisに直接依存しない。

SearchController は Echo ContextからQuery Param、Cookie、AppContextを取得し、Usecase Inputへ変換して SearchUsecaseを呼び出す。

SearchValidatorはSearchUsecaseから呼び出す。

Repositoryには依存しない。

ArticleController

ArticleControllerは、公開済みArticleの一覧、Guest向けPreview、User向けFull Detailを返す。

Guestは記事一覧とPreviewを見られる。全文表示がログイン必須の場合は、Full Detailを認証APIとして分ける。

Controller	関数名	HTTP	認証	主なUsecase	役割
ArticleController	ListArticles	GET /articles	不要	ArticleUsecase.ListArticles	Article-
ArticleController	GetArticlePreview	GET /articles/:slug/preview	不要	ArticleUsecase.GetArticlePreview	GuestPreview

Controller	関数名	HTTP	認証	主なUsecase	役割
<code>ArticleController</code>	<code>GetArticleDetail</code>	<code>GET /articles/:slug</code>	必須	<code>ArticleUsecase.GetArticleDetail</code>	User向 細

ListArticles

項目	内容
Query	<code>limit</code> , <code>offset</code> , <code>category</code> , <code>sort</code>
Input	<code>ListArticlesInput</code>
Response	<code>ArticleListResponse</code>
成功時	<code>200 OK</code>

GetArticlePreview

項目	内容
Path Param	<code>slug</code>
Input	<code>GetArticlePreviewInput</code>
Response	<code>ArticlePreviewResponse</code>
成功時	<code>200 OK</code>
用途	Guestに一部内容を表示し、続きはログインへ誘導する

GetArticleDetail

項目	内容
Path Param	<code>slug</code>
Input	<code>GetArticleDetailInput</code>
Response	<code>ArticleDetailResponse</code>
成功時	<code>200 OK</code>
失敗例	unauthorized, article not found

EventController

EventControllerは、行動ログ記録APIを担当。

EventControllerはevent_typeごとの業務判断をしない。

event_typeごとの必須項目、対象存在確認、重複排除、記録可否はEventUsecaseで判断する。

Controller	関数名	HTTP	認証	主なUsecase	役割
<code>EventController</code>	<code>RecordEvent</code>	<code>POST /events</code>	UserまたはGuest	<code>EventUsecase.RecordEvent</code>	行動ログ記録

RecordEvent

項目	内容
Request	<code>RecordEventRequest</code>
Input	<code>RecordEventInput</code>
Response	<code>RecordEventResponse</code>
成功時	<code>201 Created</code> または <code>204 No Content</code>
失敗例	invalid event_type, invalid content_type, actor required, rate limited

RecordEventで扱う主なevent_type

event_type	作成する場所	理由
<code>content_view</code>	<code>EventUsecase.RecordEvent</code>	詳細閲覧イベント
<code>impression</code>	<code>EventUsecase.RecordEvent</code>	表示イベント

event_type	作成する場所	理由
stay	EventUsecase.RecordEvent	滞在イベント
click	EventUsecase.RecordEvent	通常クリック
re_search	EventUsecase.RecordEvent	検索条件変更イベント。POST /events + CSRFで記録する
save	SavedItemUsecase	保存成功後だけ記録すべき
rating	RatingUsecase	評価成功後だけ記録すべき
modal_impression	ModalUsecase	モーダル表示可否判定と一体
modal_click	ModalUsecase	modal_display_logs更新と一体
modal_close	ModalUsecase	close記録・抑制判定と一体

Controller	API	直接受け付けるevent_type	受け付けないevent_type
EventController.RecordEvent	POST /events	content_view, impression, stay, click, re_search	save, rating, modal_impression, modal_click, modal_close
SearchController.SearchBeans/SearchArticles	GET /search/*	なし	re_searchを内部作成しない
SaveController.SaveContent	POST /saved	saveを内部作成	clientから save event を受け取らない
RatingController.RateContent	POST /ratings	ratingを内部作成	clientから rating event を受け取らない

RecommendationController

RecommendationControllerは、UserまたはGuestの行動データ・興味スコア・ランキング指標をもとに、推薦一覧を返す。推薦スコア計算、並び替え、推薦理由生成はUsecaseで行う。

Controller	関数名	HTTP	認証	主なUsecase
RecommendationController	ListRecommendations	GET /recommendations	UserまたはGuest	RecommendationUsecase.ListRecommendations

ListRecommendations

項目	内容
Query	limit, content_type, origin, roast_level, category など
Input	ListRecommendationsInput
Response	RecommendationListResponse
成功時	200 OK
失敗例	actor required, invalid query, rate limited

項目	内容
Method	GET
Path	/recommendations
Query	limit, content_type, exclude_viewed, reason_required, context
認証	任意
Controllerの処理	Queryを取得し、AuthUserIDまたはGuestSessionIDとともにRecommendationUsecase.ListRecommendationsへ渡す
成功レスポンス	200 OK

RecommendationControllerでは、以下の検索条件を受け取らない。

禁止Query	理由
keyword	SearchUsecaseの責務のため
origin	SearchUsecaseの責務のため
roast_level	SearchUsecaseの責務のため
acidity	SearchUsecaseの責務のため

禁止Query	理由
bitterness	SearchUsecaseの責務のため
flavor	SearchUsecaseの責務のため
aroma	SearchUsecaseの責務のため
body	SearchUsecaseの責務のため
category	SearchUsecaseの責務のため

ModalController

```

type AuthUsecase interface {
    Signup(ctx context.Context, input SignupInput) (SignupOutput, error)
    Login(ctx context.Context, input LoginInput) (LoginOutput, error)
    RefreshAccessToken(ctx context.Context, input RefreshAccessTokenInput) (RefreshAccessTokenOutput, error)
    Logout(ctx context.Context, input LogoutInput) error
    LogoutAll(ctx context.Context, input LogoutAllInput) error
    GetCurrentUser(ctx context.Context, input GetCurrentUserInput) (CurrentUserOutput, error)
}

```

ModalControllerは、推薦モーダルの表示候補取得、クリック、閉じる操作を担当。

モーダル表示可否、表示候補選定、保存済み除外、直近表示済み除外、表示理由生成はModalUsecaseで行う。

ControllerはEventUsecaseを直接呼ばない。

Controller	関数名	HTTP	認証	主なUsecase	役割
ModalController	GetRecommendationModal	GET /recommendation-modal	UserまたはGuest	ModalUsecase.GetRecommendationModal	推薦モーダル候補取得
ModalController	RecordModalClick	POST /recommendation-modal/:id/click	UserまたはGuest	ModalUsecase.RecordModalClick	モーダルクリック記録
ModalController	RecordModalClose	POST /recommendation-modal/:id/close	UserまたはGuest	ModalUsecase.RecordModalClose	モーダル閉じ記録

RecordModalClick / RecordModalClose は、modal_display_log_idだけを信用しない。

ControllerはAppContextからUserIDまたはGuestSessionIDを取得し、Usecase Inputへ渡す。

本人の表示ログかどうかはModalUsecaseで確認する。

GetRecommendationModal

項目	内容
Query	page_path, trigger, content_type, content_id など
Input	GetRecommendationModalInput
Response	RecommendationModalResponse
成功時	200 OK
候補なし	200 OK + show_modal: false
失敗例	actor required, invalid trigger, rate limited

RecordModalClick

項目	内容
Path Param	id
Input	RecordModalClickInput
Response	RecordModalActionResponse
成功時	204 No Content

RecordModalClose

項目	内容
Path Param	<code>id</code>
Input	<code>RecordModalCloseInput</code>
Response	<code>RecordModalActionResponse</code>
成功時	<code>204 No Content</code>

SaveController

SaveControllerは、ログインユーザーによるBean / Articleの保存、保存解除、保存一覧取得を担当。

Guestは保存できないため、RouterでAuthMiddlewareを必須。

保存可能か、対象が存在するか、すでに保存済みかはSavedItemUsecaseで判断。

Controller	関数名	HTTP	認証	主なUsecase	役割
SaveController	SaveContent	POST /saved	必須	SavedItemUsecase.SaveContent	保存
SaveController	UnsaveContent	DELETE /saved	必須	SavedItemUsecase.UnsaveContent	保存解除
SaveController	ListSavedContents	GET /saved	必須	SavedItemUsecase.ListSavedContents	保存一覧

SaveContent

項目	内容
Request	<code>SaveContentRequest</code>
Input	<code>SaveContentInput</code>
Response	<code>SavedContentResponse</code>
成功時	<code>201 Created</code>
失敗例	unauthorized, content not found, already saved, rate limited

UnsaveContent

項目	内容
Request	<code>UnsaveContentRequest</code>
Input	<code>UnsaveContentInput</code>
Response	なし
成功時	<code>204 No Content</code>
失敗例	unauthorized, content not found

ListSavedContents

項目	内容
Query	<code>limit</code> , <code>offset</code> , <code>content_type</code>
Input	<code>ListSavedContentsInput</code>
Response	<code>SavedContentListResponse</code>
成功時	<code>200 OK</code>

RatingController

RatingControllerは、ログインユーザーによるGood / Bad評価、自分の評価取得を担当。

評価可能か、対象が存在するか、再評価時に更新するかはRatingUsecaseで判断。

Controller	関数名	HTTP	認証	主なUsecase	役割
RatingController	RateContent	POST /ratings	必須	RatingUsecase.RateContent	評価
RatingController	GetMyRating	GET /ratings/me	必須	RatingUsecase.GetMyRating	自分の評価取得

RateContent

項目	内容
Request	<code>RateContentRequest</code>
Input	<code>RateContentInput</code>
Response	<code>RatingResponse</code>
成功時	<code>200 OK</code> または <code>201 Created</code>
失敗例	unauthorized, invalid score, content not found

GetMyRating

項目	内容
Method	GET
Path	/ratings/me
Query	rank_target_id
認証	必須
Controllerの処理	rank_target_id を取得し、AuthUserID とともに RatingUsecase.GetMyRating に渡す
成功レスポンス	200 OK
未評価	404 Not Found または rating: null のどちらかに統一する

項目	内容
Query	<code>content_type</code> , <code>content_id</code>
Input	<code>GetMyRatingInput</code>
Response	<code>RatingResponse</code>
成功時	<code>200 OK</code>
未評価	<code>200 OK</code> + <code>rating: null</code>

AdminBeanController

AdminBeanControllerは、AdminによるBean作成、更新、公開、非公開を担当。

Admin権限の入口チェックはAdminMiddlewareで行う。

Beanの保存可否、公開可否、RankTarget同期、AuditLog記録はAdminBeanUsecaseで判断する。

Controller	関数名	HTTP	認証	主なUsecase	役割
<code>AdminBeanController</code>	<code>CreateBean</code>	<code>POST /admin/beans</code>	Admin	<code>AdminBeanUsecase.CreateBean</code>	Bean作
<code>AdminBeanController</code>	<code>UpdateBean</code>	<code>PUT /admin/beans/:id</code>	Admin	<code>AdminBeanUsecase.UpdateBean</code>	Bean更
<code>AdminBeanController</code>	<code>PublishBean</code>	<code>PATCH /admin/beans/:id/publish</code>	Admin	<code>AdminBeanUsecase.PublishBean</code>	Bean公
<code>AdminBeanController</code>	<code>UnpublishBean</code>	<code>PATCH /admin/beans/:id/unpublish</code>	Admin	<code>AdminBeanUsecase.UnpublishBean</code>	Bean非

AdminArticleController

AdminArticleControllerは、AdminによるArticle作成、更新、公開、非公開を担当。

Admin権限の入口チェックはAdminMiddlewareで行う。

Articleの保存可否、公開可否、RankTarget同期、AuditLog記録はAdminArticleUsecaseで判断する。

Controller	関数名	HTTP	認証	主なUsecase
<code>AdminArticleController</code>	<code>CreateArticle</code>	<code>POST /admin/articles</code>	Admin	<code>AdminArticleUsecase.CreateArtic</code>
<code>AdminArticleController</code>	<code>UpdateArticle</code>	<code>PUT /admin/articles/:id</code>	Admin	<code>AdminArticleUsecase.UpdateArtic</code>
<code>AdminArticleController</code>	<code>PublishArticle</code>	<code>PATCH /admin/articles/:id/publish</code>	Admin	<code>AdminArticleUsecase.PublishArti</code>
<code>AdminArticleController</code>	<code>UnpublishArticle</code>	<code>PATCH /admin/articles/:id/unpublish</code>	Admin	<code>AdminArticleUsecase.UnpublishAr</code>

AdminRelationController

AdminRelationControllerは、BeanとArticleの関連付けを担当。

Bean詳細で関連記事を最大3件表示するため、関連付けの作成、削除、表示順更新を扱う。

関連対象の存在確認、重複確認、表示順の整合性確認、AuditLog記録はAdminRelationUsecaseで判断する。

Controller	関数名	HTTP	認証	主なUsecase
AdminRelationController	CreateBeanArticleRelation	POST /admin/bean-articles	Admin	AdminRelationUsecase.CreateBeanArtic
AdminRelationController	DeleteBeanArticleRelation	DELETE /admin/bean-articles	Admin	AdminRelationUsecase.DeleteBeanArtic
AdminRelationController	UpdateBeanArticleOrder	PATCH /admin/bean-articles/order	Admin	AdminRelationUsecase.UpdateBeanArtic

AdminBatchController

AdminBatchControllerは、Adminによるランキング再計算バッチの手動実行を担当。

実際のランキング再計算、InterestProfile再計算、Redis Lock、batch_runs記録、audit_logs記録はBatchUsecaseで行う。

Controller	関数名	HTTP	認証	主なUsecase	役割
AdminBatchController	RunRankingBatch	POST /admin/batch/ranking	Admin	AdminBatchUsecase.RunRankingBatch に依存。内部でRanking/Interest batchを呼ぶ	ランキン チを手動 る

項目	内容
Method	POST
Path	/admin/batch/ranking
Request Body	なし
認証	Admin必須
処理	RankingBatchUsecase.Run を実行する
補足	InterestProfile更新も同時に必要な場合は、Usecase内部でRankingBatch後にInterestBatchを呼ぶ
成功レスポンス	202 Accepted または 200 OK に統一する

AdminAuditController

AdminAuditControllerは、Adminによる監査ログ確認を担当。

監査ログの検索条件、閲覧権限、取得範囲はUsecaseで判断。

Controller	関数名	HTTP	認証	主なUsecase	役割
AdminAuditController	ListAuditLogs	GET /admin/audit-logs	Admin	AdminAuditUsecase.ListAuditLogs	監査ログ一覧取得

HealthController

HealthControllerは、アプリの生存確認と依存先接続確認を担当。

/health はアプリが起動しているかを返す。

/ready はDBやRedisに接続できるかを確認。

Controller	関数名	HTTP	認証	主なUsecase	役割
HealthController	Health	GET /health	不要	なし	アプリの生存確認

Controller	関数名	HTTP	認証	主なUsecase	役割
HealthController	Ready	GET /ready	不要	HealthUsecase.Ready またはInfra確認	DB/Redis接続確認

Controller Interface設計

Controllerが依存するUsecaseはinterfaceで受け取る。

Controllerは具象Usecase実装に直接依存しない。

```

type AuthUsecase interface {
    Signup(ctx context.Context, input SignupInput) (SignupOutput, error)
    Login(ctx context.Context, input LoginInput) (LoginOutput, error)
    RefreshAccessToken(ctx context.Context, input RefreshAccessTokenInput) (RefreshAccessT
okenOutput, error)
    Logout(ctx context.Context, input LogoutInput) error
    LogoutAll(ctx context.Context, input LogoutAllInput) error
    GetCurrentUser(ctx context.Context, input GetCurrentUserInput) (CurrentUserOutput, err
or)
}

type EventUsecase interface {
    RecordEvent(ctx context.Context, input RecordEventInput) (RecordEventOutput, error)
}

type RecommendationUsecase interface {
    ListRecommendations(ctx context.Context, input ListRecommendationsInput) (Recommendi
onListOutput, error)
}

type ModalUsecase interface {
    GetRecommendationModal(ctx context.Context, input GetRecommendationModalInput) (Recomm
endationModalOutput, error)
    RecordModalClick(ctx context.Context, input RecordModalClickInput) error
    RecordModalClose(ctx context.Context, input RecordModalCloseInput) error
}

type SavedItemUsecase interface {
    SaveContent(ctx context.Context, input SaveContentInput) (SavedContentOutput, error)
    UnsaveContent(ctx context.Context, input UnsaveContentInput) error
    ListSavedContents(ctx context.Context, input ListSavedContentsInput) (SavedContentList
Output, error)
}

type SearchUsecase interface {
    SearchBeans(ctx context.Context, input SearchBeansInput) (SearchBeanListOutput, error)
    SearchArticles(ctx context.Context, input SearchArticlesInput) (SearchArticleListOutpu
t, error)
}

type RatingUsecase interface {
    RateContent(ctx context.Context, input RateContentInput) (RatingOutput, error)
    GetMyRating(ctx context.Context, input GetMyRatingInput) (RatingOutput, error)
}

type AdminBeanUsecase interface {
    CreateBean(ctx context.Context, input CreateBeanInput) (BeanOutput, error)
    UpdateBean(ctx context.Context, input UpdateBeanInput) (BeanOutput, error)
}

```

```

    PublishBean(ctx context.Context, input PublishBeanInput) error
    UnpublishBean(ctx context.Context, input UnpublishBeanInput) error
}

type AdminArticleUsecase interface {
    CreateArticle(ctx context.Context, input CreateArticleInput) (ArticleOutput, error)
    UpdateArticle(ctx context.Context, input UpdateArticleInput) (ArticleOutput, error)
    PublishArticle(ctx context.Context, input PublishArticleInput) error
    UnpublishArticle(ctx context.Context, input UnpublishArticleInput) error
}

type AdminRelationUsecase interface {
    CreateBeanArticleRelation(ctx context.Context, input CreateBeanArticleRelationInput) error
    DeleteBeanArticleRelation(ctx context.Context, input DeleteBeanArticleRelationInput) error
    UpdateBeanArticleOrder(ctx context.Context, input UpdateBeanArticleOrderInput) error
}

type AdminAuditUsecase interface {
    ListAuditLogs(ctx context.Context, input ListAuditLogsInput) (AuditLogListOutput, error)
}

type AdminBatchUsecase interface {
    RunRankingBatch(ctx context.Context, input RunRankingBatchInput) (RankingBatchOutput, error)
}

```

Error Response形式

全Controllerは、共通のError Response形式で返す。

```

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "入力値が不正です",
    "request_id": "req_xxx"
  }
}

```

Usecase Error	HTTP	API Code
ErrInvalidInput	400	INVALID_REQUEST
ErrValidationError	422	VALIDATION_ERROR
ErrUnauthorized	401	UNAUTHORIZED
ErrForbidden	403	FORBIDDEN
ErrNotFound	404	NOT_FOUND
ErrConflict	409	CONFLICT
ErrRateLimited	429	RATE_LIMITED
ErrSecurityIncident	401	SECURITY_INCIDENT
ErrInternal	500	INTERNAL_ERROR

Controllerでやってはいけないこと

NG	理由
Repositoryを直接呼ぶ	Usecaseを飛ばして責務が崩れる
GORMを直接使う	ControllerがDB実装に依存する
Redis Clientを直接使う	RateLimitやLockの責務が崩れる
保存可否を書く	SavedItemUsecaseの責務
評価可否を書く	RatingUsecaseの責務
ランキング計算を書く	RankingUsecaseの責務
モーダル表示可否を書く	ModalUsecaseの責務
RefreshToken rotationを書く	AuthUsecaseの責務
reuse検知を書く	AuthUsecaseの責務
user_idをRequest Bodyから受け取る	なりすまし可能
guest_session_idをRequest Bodyから受け取る	他人のGuest行動として記録できてしまう
HTTP statusをUsecaseへ渡す	UsecaseがHTTPに依存する
Echo ContextをUsecaseへ渡す	Clean Architectureが崩れる
any型を使ってAppContextを取り出す	型安全性が落ちる